
SISTEMA DE CONTROL PARA MISIONES DE RESCATE Y EXTRACCIÓN DE RECURSOS EN CIUDADES EN CONFLICTO: UNA IMPLEMENTACIÓN CON ESTRUCTURAS DE DATOS PROPIAS Y BÚSQUEDA EN ANCHURA

202504933 – Angel David Escobar Garcia

Resumen

Este ensayo describe el diseño e implementación de un sistema de control para la empresa ficticia Chapín Warriors, S. A., orientado a coordinar misiones de rescate y extracción de recursos en ciudades afectadas por conflictos. El sistema procesa archivos de configuración en formato XML que describen la topología de una ciudad como una malla bidimensional de celdas, cada una clasificada como camino, punto de entrada, unidad militar, unidad civil o recurso. A partir de esta información, el sistema determina rutas óptimas mediante el algoritmo de búsqueda en anchura (BFS), diferenciando dos comportamientos: evitar por completo unidades militares en misiones de rescate, o enfrentarlas cuando la capacidad de combate del robot lo permite en misiones de extracción. La solución se implementó completamente en C#, utilizando tipos de dato abstracto desarrollados desde cero en lugar de las colecciones nativas del lenguaje, y aplicando los cuatro pilares de la programación orientada a objetos. Los resultados de cada misión se visualizan gráficamente mediante la herramienta Graphviz, generando automáticamente representaciones de la ciudad con la ruta recorrida resaltada.

Palabras clave

búsqueda en anchura, tipos de dato abstracto, programación orientada a objetos, Graphviz, rutas óptimas

Abstract

This essay describes the design and implementation of a control system for the fictional company Chapín Warriors, S. A., aimed at coordinating rescue and resource extraction missions in cities affected by conflict. The system processes XML configuration files describing a city's topology as a two-dimensional grid of cells, each classified as a path, entry point, military unit, civilian unit, or resource. Based on this information, the system determines optimal routes using the breadth-first search (BFS) algorithm, distinguishing two behaviors: completely avoiding military units during rescue missions, or engaging them when the robot's combat capacity allows it during extraction missions. The solution was fully implemented in C#, using abstract data types built from scratch instead of the language's native collections, and applying the four pillars of object-oriented programming. Mission results are visualized graphically using Graphviz, automatically generating city representations with the traveled route highlighted.

Keywords

breadth-first search, abstract data types, object-oriented programming, Graphviz, optimal routing

Introducción

En contextos de conflictos, la capacidad de planificar rutas seguras y eficientes hacia objetivos específicos es un problema recurrente, con aplicaciones que van desde la robótica de rescate hasta la logística militar. Este ensayo aborda el desarrollo de un sistema de control capaz de recibir la representación digital de una ciudad y, a partir de ella, calcular rutas viables para dos tipos de misión con restricciones distintas: rescate de civiles, que exige evadir todo peligro militar, y extracción de recursos, que permite enfrentar obstáculos cuando existe suficiente capacidad de combate. El desarrollo se sustenta en tres pilares técnicos: estructuras de datos construidas íntegramente por el autor, el algoritmo de búsqueda en anchura para el cálculo de rutas, y la generación automática de reportes visuales. A continuación se detalla el diseño y la implementación de cada componente.

Desarrollo del tema

El desarrollo del sistema se sustenta en cuatro componentes técnicos que se abordan a continuación: las estructuras de datos propias que sostienen toda la información del sistema, el algoritmo de búsqueda utilizado para calcular rutas, la aplicación de los principios de la programación orientada a objetos, y finalmente el mecanismo de generación de reportes visuales.

a. Tipos de Dato Abstracto propios

El enunciado del proyecto exige explícitamente prescindir de las colecciones nativas de C#, obligando a construir estructuras de memoria dinámica desde los fundamentos. Se implementó una clase genérica `Nodo <T>`, compuesta por un campo de dato y un puntero al nodo siguiente, sobre la cual se construyeron dos estructuras: `ListaSimple <T>`, una lista enlazada simple con operaciones de inserción al final, acceso por índice, búsqueda y eliminación por condición; y `Cola <T>`, una estructura FIFO con las operaciones clásicas de encolar y desencolar. Estas estructuras genéricas permitieron representar toda la información del dominio de filas de la ciudad, celdas, robots, y los nodos de exploración del algoritmo de búsqueda, sin recurrir en ningún momento a colecciones del framework. La genericidad `<T>` fue clave para evitar duplicar código: la misma `ListaSimple <T>` almacena indistintamente ciudades, robots o celdas, según el tipo con el que se instancie.

b. Algoritmo de búsqueda en anchura (BFS)

Para el cálculo de rutas se seleccionó el algoritmo de búsqueda en anchura, adecuado porque garantiza encontrar el camino más corto en grafos no ponderados. El algoritmo explora la ciudad como un grafo implícito, donde cada celda es un nodo conectado a sus cuatro vecinos ortogonales. Se implementó utilizando la `Cola <T>` propia para mantener la frontera de exploración, y una matriz de celdas visitadas para evitar ciclos. La diferencia entre los dos tipos de misión reside en la condición de expansión: en una misión de rescate, cualquier celda de tipo unidad militar queda excluida de forma absoluta; en una misión de extracción, una celda militar puede visitarse si la capacidad de combate acumulada del robot supera la capacidad de la unidad militar, en cuyo caso dicha capacidad se reduce en la diferencia correspondiente antes de continuar la búsqueda. Cada nodo de exploración conserva una referencia a su nodo padre, lo que permite reconstruir la ruta completa una vez alcanzado el objetivo, recorriendo la cadena de padres desde el destino hasta el origen.

c. Programación orientada a objetos

El sistema aplica los cuatro pilares de la programación orientada a objetos. La abstracción se refleja en la clase `Robot`, declarada de forma abstracta, que captura el comportamiento común de toda unidad robótica sin permitir instanciarla directamente. La herencia se materializa en las clases `ChapinRescue` y `ChapinFighter`, que extienden `Robot`; la segunda añade el atributo de capacidad de combate, propio únicamente de las unidades capaces de enfrentar oposición militar. El polimorfismo se aprovecha al mantener listas de tipo `Robot` que, en tiempo de ejecución, contienen instancias concretas de ambas subclases, permitiendo filtrarlas según su tipo real mediante el operador `is` de C#. Finalmente, la encapsulación se aplica

consistentemente en las clases de dominio (Celda, Ciudad), exponiendo únicamente los métodos necesarios para interactuar con la malla, como ObtenerCelda u ObtenerCeldasPorTipo, sin exponer directamente las estructuras internas de almacenamiento.

d. Generación de reportes con Graphviz

Para satisfacer el requerimiento de visualización gráfica de resultados, se optó por traducir la malla de la ciudad y la ruta calculada a una tabla HTML embebida dentro de un nodo de Graphviz, técnica que permite representar una cuadrícula regular. Cada celda de la tabla se colorea según su tipo: verde para entrada, rojo para unidad militar, azul para unidad civil, gris oscuro para recurso, negro para intransitable y las celdas pertenecientes a la ruta calculada se resaltan en color naranja. El sistema genera el archivo .dot correspondiente y, mediante la clase System.Diagnostics.Process, invoca directamente el ejecutable dot de Graphviz para producir la imagen .png final, sin requerir intervención manual del usuario. Cada reporte se almacena con un nombre único compuesto por el tipo de misión, la ciudad y una marca de tiempo, conformando un historial de todas las misiones ejecutadas durante una sesión.

En conjunto, estos cuatro componentes conforman un sistema cohesivo: las estructuras de datos propias garantizan que la información se maneje de forma eficiente y sin dependencias externas; el algoritmo de búsqueda en anchura traduce las reglas del problema en un recorrido calculable; los principios de programación orientada a objetos organizan el dominio de manera clara y extensible y la generación automática de reportes cierra el ciclo, comunicando visualmente el resultado de cada misión sin intervención manual del usuario.

Variable	Clase	Descripción
Nombre	Ciudad	Identificador único de la ciudad.
Filas, columnas	Ciudad	Dimensiones de la malla bidimensional que representa la ciudad
Tipo	Celda	Intransitable, Entrada, Camino, Civil, Recurso o Militar
Fila, Columna	Celda	Coordenadas de la celda dentro de la malla
CapacidadMilitar	Celda	Capacidad de combate de la unidad militar presente en la celda
Nombre	Robot	Identificador del robot, definido en el archivo de configuración
Capacidad	ChapinFighter	Capacidad de combate del robot; disminuye al derrotar unidades militares durante una misión
capacidadActual	NodoRuta (BFS)	Capacidad de combate restante del robot en un punto específico de la ruta explorada
Padre	NodoRuta (BFS)	Referencia al nodo anterior en la ruta, utilizada para reconstruir el camino final

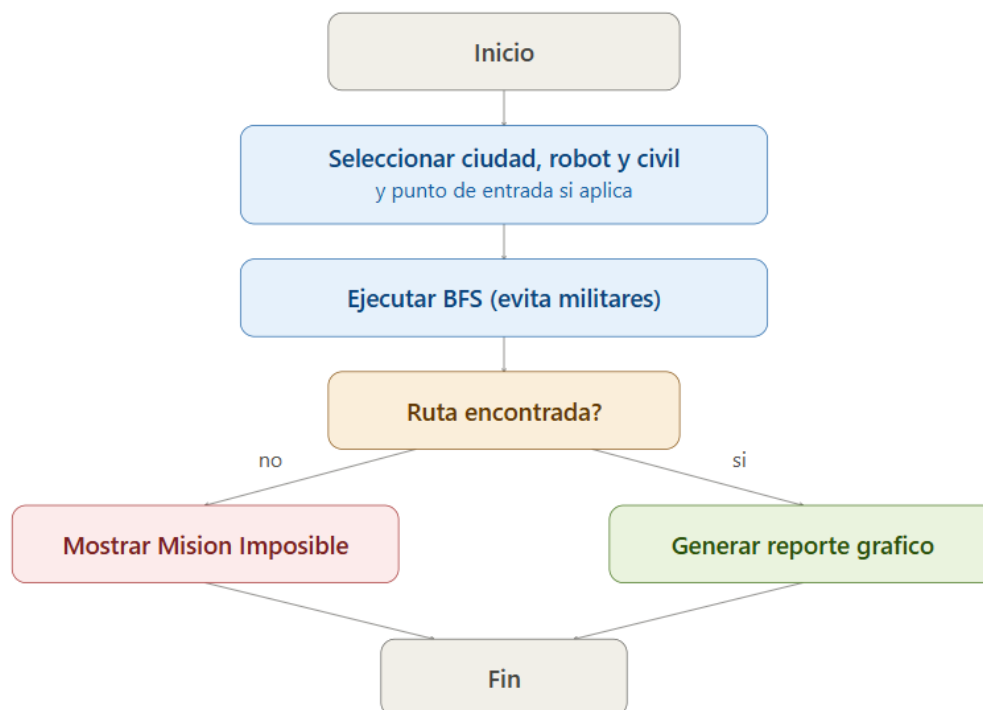
Conclusiones

El desarrollo de este sistema evidenció que la implementación de estructuras de datos propias, lejos de ser una limitación impuesta arbitrariamente, resulta formativa: obliga a comprender en profundidad el funcionamiento interno de las colecciones que normalmente se dan por sentadas en un lenguaje de alto nivel. Asimismo, se comprobó que el algoritmo de búsqueda en anchura, pese a su simplicidad conceptual, es suficientemente flexible para modelar restricciones de dominio complejas mediante ajustes en su criterio de expansión, sin alterar su estructura fundamental. La integración de Graphviz como motor de reportes demostró ser una solución práctica y de bajo costo de implementación para comunicar resultados de forma visual, particularmente valiosa en sistemas donde la salida principal es espacial por naturaleza. Como línea de trabajo futura, el sistema podría extenderse para calcular rutas óptimas considerando múltiples puntos de entrada de forma simultánea, o para soportar misiones combinadas que requieran rescatar civiles y extraer recursos en una misma incursión.

Anexos

DIAGRAMA DE ACTIVIDADES:

1.MISION DE RESCATE:



2.MISION DE RESCATE:

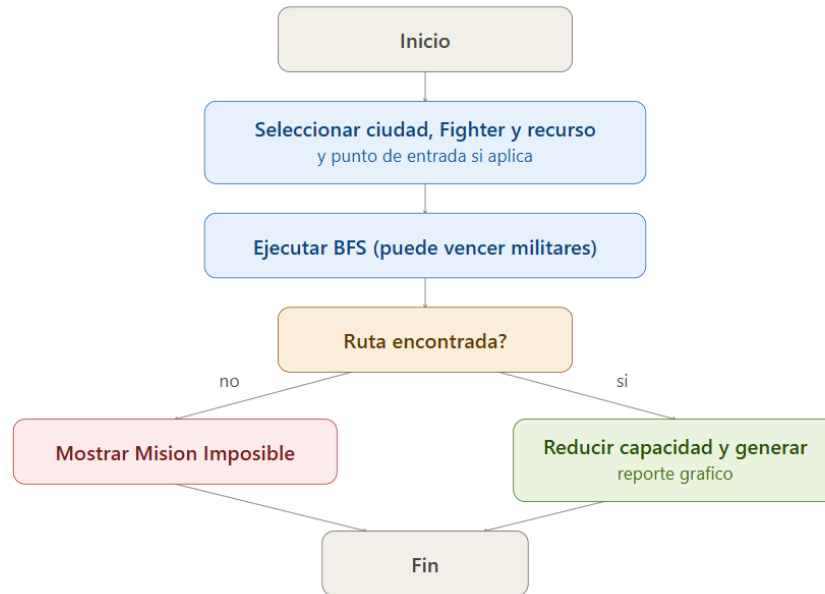


DIAGRAMA DE CLASES

